

Tái lập và MLOps tối thiểu (Reproducibility and Minimal MLOps)

PHỤ THUỘC B/06-chat-luong-du-lieu và B/07-chia-du-lieu-va-danh-gia (dữ liệu là thứ thay đổi nhiều nhất); B/08-dich-chuyen-phan-phoi (vì sao cần giám sát sau triển khai); D/19/02-pytorch-thanh-thao (seed, chế độ tắt định); D/21-inference-va-trien-khai (mô hình đã ở production); E/24-thi-nghiem-va-ablation (một thay đổi = một config diff).

TÓM TẮT

Sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này — mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó. Chương đi qua config tập trung, theo dõi thí nghiệm, quản lý phiên bản dữ liệu, đóng gói artifact đầy đủ, các nguồn bất định trên GPU, CI cho mô hình, và giám sát drift sau triển khai. Đọc xong bạn dựng lại được một kết quả cũ chỉ từ một config, và — quan trọng hơn — biết khi nào việc dựng lại *từng chữ số* là mục tiêu sai. Nếu chỉ nhớ một điều: mục tiêu đúng không phải tái lập từng bit mà là tái lập *phân phối* kết quả, nên một con số không kèm phương sai không phải bằng chứng.

KIẾN THỨC CẦN CÓ TRƯỚC

- **seed** — số khởi tạo mọi bộ sinh ngẫu nhiên trong một lần chạy — khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed không đủ để cùng kết quả, nhưng khác seed thì chắc chắn khác kết quả.
- **config** — tệp mô tả toàn bộ siêu tham số và lựa chọn của một lần chạy, tách khỏi mã nguồn. Nếu hai lần chạy khác nhau mà không diff được config thì bạn không ablate được.
- **experiment tracking** — hệ thống ghi lại mỗi lần chạy kèm commit mã, config, phiên bản dữ liệu và các chỉ số theo thời gian, để sau này truy ngược được từ một con số về nguồn gốc của nó.
- **data versioning** — quản lý phiên bản cho dữ liệu giống như git quản lý mã: mỗi trạng thái của tập dữ liệu có một mã băm, và một lần chạy trở tới đúng mã băm đó.
- **artifact** — gói đầy đủ cần để tái hiện hoặc triển khai một mô hình: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Chỉ có checkpoint là chưa đủ.
- **tính tắt định (determinism)** — chạy lại cùng đầu vào cho ra cùng đầu ra tới từng bit. Trên GPU nó không mặc định, và ép nó phải trả giá bằng tốc độ.
- **CI cho mô hình** — một tập kiểm tra rẽ chạy tự động mỗi commit — overfit một batch, kiểm shape, kiểm export thành công, kiểm chỉ số không tụt quá ngưỡng trên một tập nhỏ cố định.
- **drift** — sự trôi dạt của phân phối dữ liệu thật so với phân phối lúc huấn luyện. Nó làm mô hình xuống cấp mà không có lỗi nào được ném ra.

CÂU HỎI MỞ ĐẦU

- Q1. Bạn cố định *seed*, chạy lại đúng mã trên đúng máy, và vẫn nhận hai con số khác nhau — ai đã đổi thứ tự phép cộng sau lưng bạn?
- Q2. Vì sao ép mọi thứ tắt định lại là một mục tiêu *sai*, dù nó nghe đúng hết thứ một kỹ sư nên làm?

- Q3. Cấu hình mới hơn cấu hình cũ 0,3 mAP. Vì sao con số đó có thể hoàn toàn vô nghĩa, và cần bao nhiêu lần chạy để nó có nghĩa?
- Q4. Sáu tháng sau bạn có checkpoint, có mã, có config — và vẫn không dựng lại được kết quả cũ. Thứ còn thiếu thường là gì?

1.1 Đặt vấn đề

Câu hỏi dẫn dắt rất cụ thể và rất quen: *sáu tháng sau, bạn cần biết chính xác cái gì đã tạo ra con số này*. Không phải “đại khái cấu hình nào”, mà là chính xác: commit nào, config nào, phiên bản dữ liệu nào, thư viện phiên bản mấy, chạy trên GPU gì. Mọi công cụ trong chương tồn tại vì trí nhớ con người không làm được việc đó, và vì thư mục `runs/exp_final_v2_real_final` không phải một hệ thống.

Nhưng chương này còn một nửa thứ hai, và nửa ấy mới là phần khó. Ngay cả khi bạn đã ghi lại mọi thứ và cố định seed, kết quả *vẫn* không lặp lại chính xác. Người đã debug bắt định trong một hệ thống C++ đa luồng biết ngay tại sao: khi thứ tự các phép toán không được xác định, kết quả số học cũng không được xác định. Trong deep learning, số nguồn gây ra điều đó nhiều hơn hẳn, và phần lớn nằm dưới tầng trừu tượng mà bạn không nhìn thấy.

Học xong chương này thì làm được gì mà trước đó không làm được? Bạn phân biệt được ba tình huống mà người ta hay gộp làm một: *có bug, có bất định nhưng vẫn đúng, và có thay đổi thật nhưng bị nhiễu che*. Ba tình huống ấy đòi ba hành động hoàn toàn khác nhau, và nhằm chúng là cách phổ biến nhất để hoặc đuổi theo một con ma, hoặc công bố một cải tiến không tồn tại.

TRỰC GIÁC

Coi mỗi con số bạn báo cáo là giá trị của một hàm băm với rất nhiều đối số: mã nguồn, config, dữ liệu, phiên bản thư viện, kiến trúc GPU, seed, và cả những thứ bạn không kiểm soát như thứ tự hoàn thành của các luồng. Kỹ thuật tái lập không phải là “làm cho hàm băm trở nên tắt định” — điều đó thường bất khả thi và luôn đắt. Nó là hai việc: *ghim càng nhiều đối số càng tốt, và đo phương sai gây ra bởi những đối số còn lại*.

Bạn đã sống qua đúng bài học này. Khi một benchmark SLAM cho kết quả khác nhau giữa các lần chạy, cách chữa không phải là ép mọi thứ tắt định rồi đi tiếp. Cách chữa là tách làm hai chế độ. Chế độ ghim chặt — một luồng, thứ tự cố định — *chỉ* dùng để cô lập bug. Chế độ chạy thật lặp nhiều lần dùng để báo cáo, kèm khoảng biến thiên. Chương này là cùng một kiến trúc suy nghĩ, áp cho một hệ thống có nhiều nguồn bất định hơn.

1.2 Ghim các đối số kiểm soát được

A. Bốn lớp phải ghim

Bốn nút thắt của việc truy ngược một con số

1 Vấn đề

siêu tham số rải rác trong code.

Giải pháp

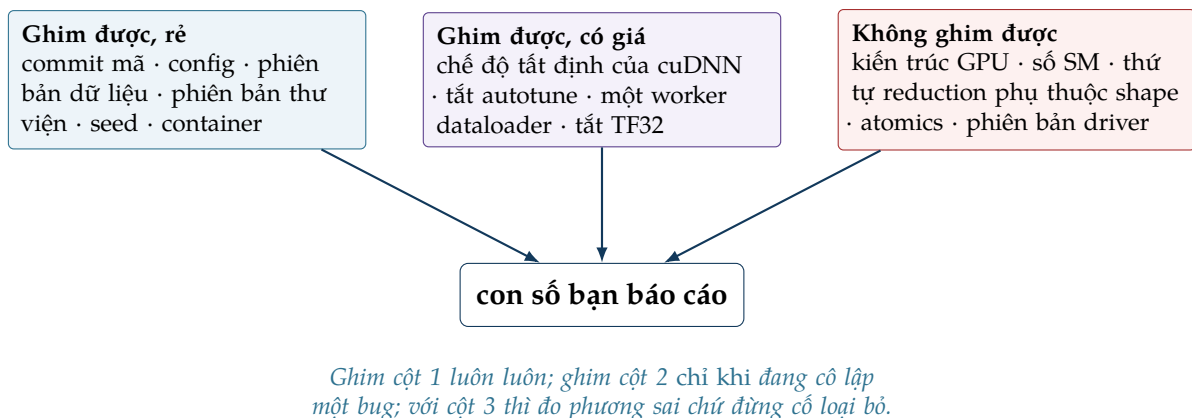
config tập trung (Hydra, hoặc YAML thuần cũng được).

Giả định

bạn tuân thủ nguyên tắc *một thay đổi = một config diff*.

Hệ quả	nếu không diff được thì không ablate được, và toàn bộ E/24-thi-nghiem-va-ablation sụp đổ.
2 Vấn đề	không biết lần chạy nào ứng với mã nào.
Giải pháp	experiment tracking gắn mỗi lần chạy với commit, config và phiên bản dữ liệu — tự động, không dựa vào kỷ luật của con người lúc hai giờ sáng.
3 Vấn đề	vấn đề bị bỏ quên nhiều nhất: dữ liệu thay đổi nhiều hơn mã.
Giải pháp	data versioning.
Giả định	mọi so sánh giữa hai thời điểm chỉ có nghĩa khi bạn biết dữ liệu có đổi hay không.
Vấn đề mới	không có nó thì một cải tiến 2 điểm có thể chỉ là kết quả của việc ai đó dọn lại nhân tuần trước.
4 Vấn đề	checkpoint không đủ để chạy lại.
Giải pháp	lưu artifact đầy đủ: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý.
Hệ quả	đúng cái tiền xử lý mà chương 21 đã chỉ ra là nguyên nhân tổn giờ người nhất khi export.

Hình 1.1 liệt kê toàn bộ đối số của hàm bấm ấy và chia chúng theo mức độ bạn kiểm soát được. Cách đọc hình: cột trái ghim được bằng công cụ, cột giữa ghim được nhưng phải trả giá, cột phải không ghim được và chỉ có thể đo phương sai.



Hình 1.1. “Chữ ký” của một con số. Sai lầm thường gặp là dồn công sức vào cột 2 — ép tắt định bằng mọi giá — trong khi cột 1 còn chưa ghim đủ và cột 3 thì vốn không thể ghim.

1.3 Các nguồn bất định — và vì sao chế độ tắt định không đủ

Đây là phần cốt lõi của chương và là phần nổi thẳng vào kinh nghiệm bạn đã có. Bảng 1.1 liệt kê các nguồn theo cơ chế, vì hiểu cơ chế thì bạn tự suy ra được nguồn mới khi gặp thư viện lạ.

Bảng 1.1. Các nguồn bất định trong huấn luyện GPU, theo cơ chế.

Nguồn	Cơ chế	Ghim thế nào	Còn lại gì sau khi ghim
Autotune của cuDNN	Chọn kernel bằng cách <i>bấm giờ</i> lúc chạy; máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác	<code>cuda.nv.benchmark = False</code>	Chậm hơn ở các shape cố định
Atomics dấu phẩy động	<code>atomicAdd</code> cộng theo thứ tự luồng hoàn thành; cộng số thực không kết hợp nên tổng khác nhau ở bit cuối	<code>use_deterministic_algorithms(True)</code>	Một số op không có bản tắt định → ném lỗi thay vì chạy
Thứ tự reduction	Chia khối phụ thuộc shape và số SM của GPU	Không ghim được	Đổi GPU là đổi kết quả, kể cả khi mọi thứ khác giống hệt
TF32 và fp16	Giảm số bit phân định trị của phép nhân ma trận; kết quả đúng nhưng khác	Tắt TF32, dùng fp32	Chậm hơn nhiều
DataLoader nhiều worker	Mỗi worker có luồng ngẫu nhiên riêng; số worker đổi là chuỗi augmentation đổi	Cố định số worker và seed từng worker	Không còn tự do chỉnh hiệu năng
Thứ tự duyệt tập hợp	Duyệt một set các đối tượng Python, hay <code>glob</code> không <code>sort()</code> , cho thứ tự phụ thuộc địa chỉ bộ nhớ hoặc hệ tệp	Luôn <code>sorted()</code> , luôn khoá theo id ổn định	Không mất gì — đây là lỗi thuần tuý, nên sửa

Dòng cuối đáng dừng lại vì nó là *cùng một lỗi* mà bạn đã gặp ở phía SLAM: một tập hợp được sắp theo địa chỉ con trỏ thì thứ tự duyệt phụ thuộc vào cách bộ cấp phát tình cờ xếp bộ nhớ trong lần chạy đó. Trong C++ nó là `std::set<T*>`; trong Python nó là một set các đối tượng không có `__hash__` tùy biến, hay một `os.listdir` chưa sắp. Điểm chung, và cũng là điểm đáng nhớ nhất: *đây không phải bất định số học mà là một bug*, nên nó không thuộc cột “không ghim được” — nó phải được sửa, và sửa xong thì không mất gì cả.

Từ đó rút ra phát biểu trung thực về giới hạn: tái lập chính xác tới từng bit giữa hai máy khác GPU thường *không khả thi*, vì thứ tự reduction phụ thuộc số SM và cách chia khối. Chế độ tắt định của PyTorch giải quyết được các nguồn trong cùng một máy, không giải quyết được nguồn giữa các máy. Mục tiêu đúng vì thế không phải tái lập từng chữ số mà là tái lập *phân phối* kết quả.

→ **hệ quả của D/19/02** — chế độ tắt định của PyTorch chỉ chặn được các nguồn trong cùng một máy, nên nó mua một đảm bảo hẹp hơn nhiều so với cái tên gợi ra.

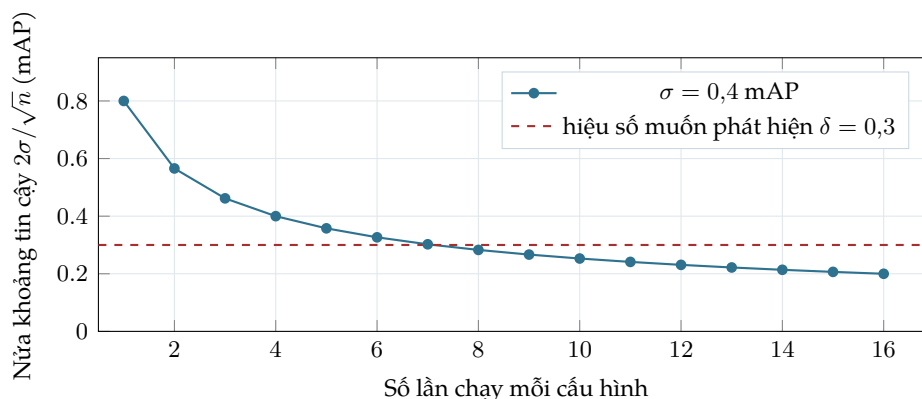
A. Ví dụ nhỏ nhất: bao nhiêu lần chạy thì đủ

Giả sử độ lệch chuẩn giữa các seed của chỉ số bạn quan tâm là $\sigma = 0,4$ điểm mAP — một giá trị không hiếm với dữ liệu vừa. Bạn thấy cấu hình mới hơn cấu hình cũ 0,3 điểm trong một lần chạy mỗi bên. Con số đó nói gì? Gần như không gì cả, vì nó nhỏ hơn cả độ lệch chuẩn của nhiễu.

Chạy n lần mỗi bên thì sai số chuẩn của trung bình là $\sigma/\sqrt{n}$. Muốn khoảng ± 2 sai số chuẩn hẹp hơn hiệu số δ mà bạn muốn phát hiện, cần

$$\frac{2\sigma}{\sqrt{n}} < \delta \iff n > \left(\frac{2\sigma}{\delta}\right)^2.$$

Ý nghĩa: độ nhạy của thí nghiệm tăng theo *căn* số lần chạy, nên muốn phát hiện hiệu số nhỏ đi một nửa thì phải chạy nhiều gấp bốn. Thay số: với $\sigma = 0,4$ và $\delta = 0,3$, ta cần $n > (0,8/0,3)^2 = 7,1$, tức tối thiểu tám lần chạy mỗi cấu hình. Nếu không đủ ngân sách cho tám lần chạy, kết luận đúng không phải “chạy một lần rồi tin”. Kết luận đúng là “hiệu số 0,3 nằm ngoài khả năng phân giải của tôi”. Đó là một kết luận hợp lệ và đáng viết ra. Hình 1.2 cho thấy vì sao.



Hình 1.2. Độ nhảy của một so sánh tăng theo căn bậc hai số lần chạy. Hai đường cắt nhau ở $n \approx 7$: dưới ngưỡng đó, mọi kết luận về hiệu số 0,3 mAP đều là đọc nhiều.

1.4 CI cho mô hình và giám sát sau triển khai

A. Bốn kiểm tra rẻ chạy mỗi commit

Mã học máy có một đặc điểm khó chịu: nó hiếm khi hỏng bằng cách ném lỗi. Vì vậy CI cho mô hình không thể là “chạy unit test” mà phải là một tập kiểm tra nhắm vào các chế độ hỏng im lặng, và cả bốn đều phải rẻ để không ai muốn tắt chúng:

↔ **tiền đề** E/24 — không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện; CI là cách ép hai điều kiện đó thành thói quen.

1. **Overfit một batch.** Cho mô hình học thuộc đúng một batch nhỏ; loss phải xuống gần không trong vài chục bước. Nếu không, có lỗi ở gradient, ở nhãn, hoặc ở chỗ nối — và bạn biết ngay trong một phút thay vì sau một đêm huấn luyện.
2. **Kiểm shape và dtype** ở mọi ranh giới khối, đặc biệt sau khi đổi độ phân giải hoặc số lớp.
3. **Kiểm export thành công** sang định dạng triển khai, và so khớp đầu ra với một mẫu cố định. Điều này biến cả chương 21 thành một bài kiểm tra tự động thay vì một sự kiện đau đớn mỗi quý.
4. **Kiểm chỉ số không tụt quá ngưỡng** trên một tập nhỏ cố định. Ngưỡng phải đặt dựa trên phương sai giữa các seed đã đo, nếu không bạn sẽ có một bài kiểm tra hỏng ngẫu nhiên — và một bài kiểm tra hỏng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.

B. Sau triển khai: drift và vòng lặp dữ liệu

Mô hình xuống cấp sau khi triển khai mà không có lỗi nào được ném ra, vì thế giới đổi còn trọng số thì không. Hai việc tối thiểu. Thứ nhất là **giám sát drift**: theo dõi phân phối đầu vào và phân phối đầu ra, chứ không chỉ độ chính xác. Lý do là chân trị thường đến muộn hoặc không bao giờ đến. Thứ hai là **thu thập lỗi làm dữ liệu mới**, tức biến các trường hợp mô hình sai hoặc không chắc thành hàng đợi gắn nhãn.

Bước thứ hai là chỗ Phần D nói ngược về Phần B và đóng vòng của cả tài liệu. Dữ liệu sinh ra mô hình; mô hình chạy trong thực tế; thực tế sinh ra dữ liệu mới. Chất lượng của vòng lặp ấy, chứ không phải kiến trúc, mới quyết định hệ thống tốt lên hay xấu đi theo thời gian.

↔ **hệ quả của** B/08 — drift là dịch chuyển phân phối quan sát từ phía vận hành; thu thập lỗi làm dữ liệu mới là cách đóng vòng lại về khâu dữ liệu.

1.5 Thực hành

Repo và tài nguyên. *Config:* [facebookresearch/hydra](#). *Theo dõi thí nghiệm:* [wandb/wandb](#), [mlflow/mlflow](#), [aimhubio/aim](#). *Phiên bản dữ liệu:* [iterative/dvc](#). *Nổi đánh giá với dữ liệu:* [voxel51/fiftyone](#). *Drift:* [evidentlyai/evidently](#). Danh sách chốt tại tháng 9/2026; nhóm công cụ này thay đổi chậm hơn hầu hết phần còn lại của tài liệu, nhưng các nền tảng theo dõi thí nghiệm dạng dịch vụ thì đến và đi khá nhanh — hãy chọn thứ xuất được dữ liệu ra định dạng mở.

Câu hỏi kiểm tra hiểu. Bạn có một checkpoint từ sáu tháng trước và số đo không tái lập được — liệt kê theo thứ tự sáu thứ có thể đã đổi. Vì sao bật chế độ tắt định không đảm bảo tái lập giữa hai máy khác GPU? Với $\sigma = 0,4$ mAP, hiệu số nhỏ nhất mà bạn có thể phát hiện với năm lần chạy mỗi bên là bao nhiêu?

Mini project (vài giờ đến hai ngày). Chuyển một dự án hiện có sang Hydra cộng một hệ theo dõi thí nghiệm, rồi chạy lại một thí nghiệm cũ *chỉ từ config* và kiểm tra khớp số. Phần đáng giá nhất không phải lúc nó khớp mà lúc nó lệch: mỗi chỗ lệch chỉ ra đúng một đối số của hàm băm mà bạn tưởng đã ghim nhưng chưa. Ghi lại danh sách đó — nó là bản kiểm kê nợ kỹ thuật của chính dự án bạn.

1.6 Giới hạn và trường hợp hỏng

- **Tắt định không phải chính xác.** Một pipeline hoàn toàn tắt định vẫn có thể sai hoàn toàn; tái lập chỉ đảm bảo bạn sai một cách nhất quán. Đây là lỗi phân loại phổ biến nhất trong cả chương.
- **Chế độ tắt định che giấu chính phương sai bạn cần biết.** Nếu bạn chỉ chạy tắt định một seed, bạn không bao giờ đo được độ nhạy của kết quả với khởi tạo — và một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động [1].
- **Ghi lại mọi thứ không đồng nghĩa với truy ngược được.** Nếu artifact thiếu mã tiền xử lý hoặc thiếu phiên bản thư viện, bạn có một kho lưu trữ đầy đủ về mặt hình thức mà vô dụng về mặt thực hành.
- **Giám sát drift phát hiện thay đổi, không phát hiện thiệt hại.** Phân phối đầu vào có thể trôi mà hiệu năng không đổi, hoặc ngược lại — đứng yên mà hiệu năng vẫn tụt vì quan hệ giữa đầu vào và nhãn đã đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.
- **Chi phí vận hành là chi phí thật.** Toàn bộ hạ tầng trong chương cần người bảo trì; một hệ MLOps bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn sai.

CẠM BẮY

Hai hiểu lầm về `use_deterministic_algorithms(True)`. **Một:** nó không đảm bảo tái lập giữa hai máy khác GPU — thứ tự reduction phụ thuộc số SM và cách chia khối, cả hai đều là thuộc tính phần cứng. Kết quả là bạn trả giá tốc độ để mua một sự đảm bảo hẹp hơn bạn tưởng. **Hai:** một lần chạy tình cờ tắt định không chứng minh được gì. Nếu mô hình của bạn hôm nay không đi qua nhánh nào dùng atomics, hai lần chạy sẽ khớp; thêm một tầng dùng `scatter_add` vào tuần sau là nó hết khớp, và bạn sẽ đi tìm bug ở đúng chỗ không có bug.

CẦN NHỚ

Một con số không kèm phương sai không phải bằng chứng. Quy trình tối thiểu: ghim mọi thứ ghim được rẻ (commit, config, dữ liệu, thư viện, container), *chỉ* bật chế độ tắt định khi đang cô lập một bug, và báo cáo trung bình kèm khoảng biến thiên trên nhiều seed. Và luôn tách ba tình huống: có bug, có bất định mà vẫn đúng, có thay đổi thật bị nhiễu che — ba tình huống ấy đòi ba hành động khác nhau. [T2]

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Seed	Số khởi tạo mọi bộ sinh ngẫu nhiên: khởi tạo trọng số, thứ tự trộn dữ liệu, augmentation, dropout. Cùng seed chưa đủ để cùng kết quả; khác seed thì chắc chắn khác.
●●● Config	Tập mô tả toàn bộ siêu tham số của một lần chạy, tách khỏi mã. Nguyên tắc: một thay đổi bằng một config diff; không diff được thì không ablate được.
●● Experiment tracking	Ghi tự động mỗi lần chạy kèm commit, config và phiên bản dữ liệu. Phải tự động, vì nó tồn tại chính để bù cho trí nhớ con người lúc hai giờ sáng.
●●● Data versioning	Quản lý phiên bản cho tập dữ liệu như git cho mã. Đây là mục hay bị bỏ quên nhất, và thiếu nó thì mọi so sánh giữa hai thời điểm đều mơ hồ.
●●● Artifact	Gói đủ để tái hiện: trọng số, config, phiên bản thư viện, phiên bản dữ liệu, và mã tiền xử lý. Thiếu một mục là cả gói mất giá trị.
●●● cudnn.benchmark	Chế độ autotune: chọn kernel bằng cách bấm giờ lúc chạy. Máy đang bận thì chọn kernel khác, và kernel khác cộng theo thứ tự khác.
●●● atomicAdd	Cộng dồn song song theo thứ tự luồng hoàn thành. Cộng số thực không có tính kết hợp, nên kết quả lệch ở bit cuối giữa hai lần chạy.
●● TF32	Chế độ nhân ma trộn giảm số bit phần định trị trên GPU Ampere trở lên, bật mặc định. Nhanh hơn nhưng cho số khác fp32 thuần.
●● Vì sao chế độ tắt định không đủ khi đổi máy?	Thứ tự reduction phụ thuộc cách chia khối và số SM, tức phụ thuộc phần cứng. <code>use_deterministic_algorithms</code> chặn được nguồn trong cùng một máy, không chặn được nguồn giữa hai máy.
●● Vì sao cùng seed, cùng máy vẫn ra hai số khác nhau?	Ba nguồn chính: autotune của cuDNN, atomics dấu phẩy động, và thứ tự reduction phụ thuộc cách chia khối. Không nguồn nào nằm trong mã của bạn.
●●● Vì sao thứ tự duyệt tập hợp là bug, không phải nhiễu?	Vì nó phụ thuộc địa chỉ bộ nhớ mà bộ cấp phát tình cờ trả về, chứ không phụ thuộc phần cứng. Sửa bằng <code>sorted()</code> và khoá theo id ổn định; sửa xong không mất gì cả.
●●● Vì sao tái lập phân phối mới là mục tiêu đúng?	Vì ép trùng từng bit xuyên máy thường bất khả thi và luôn đắt. Chạy nhiều lần rồi báo cáo khoảng dao động cho bạn đúng thứ bạn cần: biết hiệu số nào là thật.
●●● $n > (2\sigma/\delta)^2$	Số lần chạy cần để phân giải một hiệu số δ khi độ lệch giữa seed là σ . Với $\sigma = 0,4$ và $\delta = 0,3$ thì cần tám lần mỗi bên.
●● CI cho mô hình	Bốn bài kiểm tra mỗi commit: overfit một batch, kiểm shape, kiểm export và so mẫu cố định, kiểm chỉ số không tụt quá ngưỡng.
●● Drift	Phân phối dữ liệu thật trôi dần khỏi phân phối lúc huấn luyện, làm mô hình xuống cấp mà không ném lỗi nào. Giám sát cả đầu vào lẫn đầu ra, vì chân trị thường đến muộn.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Checkpoint sáu tháng tuổi không tái hiện được số cũ?	Thường thiếu mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản dữ liệu. Checkpoint chỉ chứa trọng số.
•• Bài kiểm CI lúc đỏ lúc xanh mà không ai đổi gì?	Ngưỡng đặt không dựa trên phương sai giữa các seed đã đo. Một bài kiểm hòng ngẫu nhiên sẽ bị vô hiệu hoá trong hai tuần.
•• Drift báo động nhưng độ chính xác không đổi?	Phân phối đầu vào đổi mà quan hệ đầu vào–nhân chưa đổi. Cảnh báo drift là tín hiệu để đi xem, không phải kết luận.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
<code>iterative/dvc</code>	Quản lý phiên bản dữ liệu bằng mã băm, để một lần chạy trở tới đúng trạng thái tập dữ liệu	Hạ tầng; nhóm công cụ ổn định nhất trong chương
<code>hydra + mlflow / wandb / aim</code>	Config tập trung cộng truy ngược từ một con số về commit, config và phiên bản dữ liệu	Hạ tầng; nên chọn thứ xuất được dữ liệu ra định dạng mở vì các dịch vụ đến và đi khá nhanh
<code>torch.use_deterministic</code>	Hiện thực chính xác của ranh giới “ghim được có giá” so với “không ghim được” ở Hình 1.1	Tài liệu của nó liệt kê thẳng các op không có bản tắt định — đáng đọc như một bảng kiểm
<code>evidentlyai/evidently</code>	Giám sát drift phân phối đầu vào và đầu ra khi chân trị đến muộn hoặc không đến	Công cụ; ý tưởng bền hơn tên công cụ
<code>voxel51/fiftyone</code>	Nối đánh giá với dữ liệu để biến trường hợp sai thành hàng đợi gắn nhãn — bước khép vòng	Chỗ mà chương 22 gặp lại chương 6 và 7 trong một giao diện

MINI PROJECT SLAM • Ép một lần chạy SLAM có thành phần học tái lập tới từng bit • 1–2 ngày

Mục tiêu	đạt được tái lập bit-for-bit, hoặc chứng minh được vì sao không thể. Hai kết quả có giá trị ngang nhau, miễn là bạn nêu tên từng nguồn bất định chứ không nói “GPU vốn thể”.
Dữ liệu	chính pipeline của mini project chương 21, một chuỗi EuRoC, cấu hình mặc định, không đổi gì khác.
Các bước	chạy mười lần, ghi ATE mỗi lần, tính σ ban đầu. Rồi ghim từng nguồn một và sau mỗi lần ghim chạy lại năm lần để xem σ còn lại bao nhiêu: seed toàn cục; <code>sorted()</code> cho mọi lần duyệt tập hợp, <code>glob</code> và <code>listdir</code> ; một <i>worker</i> dataloader; <code> cudnn.benchmark = False; use_deterministic_algorithms(True); tắt TF32</code> . Ghi cả cái giá về thời gian của từng bước.
Xong khi	mười lần chạy cho ATE giống nhau tới chữ số cuối cùng bạn in ra. Hoặc một bảng ba cột: nguồn bất định, đã cố định được chưa, và giá phải trả hay lý do không cố định được. Lý do phải cụ thể: op nào không có bản tắt định, chỗ nào dùng <code>atomicAdd</code> , chỗ nào phụ thuộc số SM. Kèm một câu dùng được ngay: “chênh lệch ATE nhỏ hơn X cm giữa hai cấu hình là nhiều, không phải cải tiến”.
Nối với	D/22 cho hình chữ ký của một con số và bảng nguồn bất định; E/24 cho cách đọc con số σ đó khi so hai cấu hình; E/26 nếu bạn muốn xếp các mini project này vào một lộ trình học theo mục tiêu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Chủ yếu là ba thứ: autotune của cuDNN chọn kernel bằng cách bấm giờ nên máy bạn thì chọn khác; atomicAdd cộng theo thứ tự luồng hoàn thành mà cộng số thực không có tính kết hợp; và thứ tự reduction phụ thuộc cách chia khối. Không thứ nào trong ba thứ đó nằm trong mã của bạn.
- A2. Vì tất định không phải chính xác — một pipeline hoàn toàn tất định vẫn có thể sai một cách nhất quán — và vì chạy tất định một seed che mất chính phương sai bạn cần biết. Một phương pháp chỉ hoạt động với một seed là một phương pháp không hoạt động.
- A3. Vì 0,3 nhỏ hơn cả độ lệch chuẩn giữa các seed, vốn thường quanh 0,4 mAP với dữ liệu vừa. Muốn $2\sigma/\sqrt{n} < \delta$ thì cần $n > (0,8/0,3)^2 \approx 7,1$, tức tối thiểu tám lần chạy mỗi bên; không đủ ngân sách thì kết luận đúng là “hiệu số này nằm ngoài khả năng phân giải của tôi”.
- A4. Thường là mã tiền xử lý và phiên bản thư viện, kể đó là phiên bản tập dữ liệu. Checkpoint chỉ chứa trọng số; nếu cách resize hay giá trị chuẩn hoá đã đổi thì mô hình cũ chạy trên dữ liệu mới là một hệ thống khác, dù mọi tệp đều còn nguyên.

NÓI LẠI THẬT ĐƠN GIẢN

Bà bạn có một món ăn ngon. Bà đọc cho bạn công thức, bạn ghi lại, và sáu tháng sau nấu ra một món khác hẳn. Không phải bạn làm sai — chỉ là công thức không ghi loại bột nào, lò của ai, đun mấy phút ở nấc nào, và hôm ấy trời ẩm ra sao.

Đây đúng là tình cảnh của mọi người làm mô hình. Con số đẹp bạn đo hôm nay là kết quả của rất nhiều thứ, phần lớn không nằm trong tệp mã nguồn: phiên bản thư viện, thứ tự dữ liệu, loại máy, và cả những chuyện xảy ra bên trong máy mà bạn không điều khiển được. Máy tính cộng số lẻ theo thứ tự khác nhau thì ra kết quả lệch nhau ở chữ số cuối, và thứ tự đó lại phụ thuộc vào việc lúc ấy máy đang bận thế nào.

Mẹo gồm hai nửa. Nửa thứ nhất là ghi lại mọi thứ ghi được, và ghi tự động chứ đừng trông vào trí nhớ lúc hai giờ sáng: đúng bản mã nào, đúng bộ dữ liệu nào, đúng phiên bản thư viện nào. Nửa thứ hai quan trọng hơn và hay bị bỏ qua: chấp nhận rằng phần còn lại không ghim được, nên hãy nấu năm lần và biết món ăn dao động trong khoảng nào. Khi ai đó bảo cách mới ngon hơn, bạn hỏi được ngay: hơn nhiều hơn khoảng dao động đó không?

Cái giá là công sức: toàn bộ bộ máy ghi chép này cần người duy trì, và một hệ thống ghi chép bị bỏ mặc còn nguy hiểm hơn không có, vì nó tạo cảm giác an toàn giả.

Một câu: Một con số không kèm khoảng dao động không phải bằng chứng — hãy ghim mọi thứ ghim được rẻ, rồi đo phần còn lại thay vì giả vờ nó không tồn tại.

1.7 Kết nối

Chương này là điều kiện cần của E/24-thi-nghiệm-va-ablation: không có config diff được và không có phương sai đo được thì mọi ablation chỉ là kể chuyện. Nó nối ngược về B/06-chat-luong-du-lieu và B/08-dich-chuyen-phan-phoi qua vòng lặp thu thập lỗi, và chính vòng lặp đó là chỗ Phần D khép lại với Phần B. Nó mở rộng mục về tính tất định trong D/19/02-pytorch-thanh-thao từ một đoạn thành một khung suy nghĩ. Và nó là nền cho E/25-do-tin-cay-va-van-hanh: giám sát drift chỉ là một trường hợp riêng của câu hỏi lớn hơn — làm sao biết mô hình đang ở ngoài vùng nó đáng tin.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Bạn có một checkpoint sáu tháng tuổi và số đo không tái lập được — sáu thứ có thể đã đổi, xếp theo xác suất, là những gì?
2. Vì sao bật chế độ tất định không đảm bảo tái lập giữa hai máy khác GPU, và cơ chế phần cứng nào chịu trách nhiệm?

3. Thứ tự duyệt một set các đối tượng thuộc cột “không ghim được” hay cột “phải sửa”, và vì sao phân loại đó quan trọng?
4. Với $\sigma = 0,4$ mAP, cần bao nhiêu lần chạy mỗi cấu hình để phát hiện tin cậy một hiệu số 0,2 mAP, và con số đó nói gì về chi phí của việc theo đuổi các cải tiến nhỏ?
5. Vì sao “overfit một batch” là bài kiểm tra CI đáng giá nhất trong bốn bài, và nó bắt được loại lỗi nào mà kiểm shape không bắt được?
6. Giám sát drift báo động nhưng độ chính xác không đổi — điều đó có nghĩa gì, và ngược lại thì sao?

Nguồn

- [1] P. Henderson, R. Islam, P. Bachman, et al., “Deep Reinforcement Learning That Matters,” *AAAI*, 2018.